

Infrastructure as Code

(Patterns and Practice)

Fundamentals of System Security
(COMSM0122)

Learning Outcome

- Defining infrastructure
- Defining infrastructure as code
- The nature of change
- Understanding why infrastructure as code is important
- Explain the RICE principles for reviewing IaC
- IaC Platform and tools

Learning Resources

2017 IEEE/ACM 39th IEEE International Conference on Software Engineering Companion

DevOps: Introducing Infrastructure-as-Code

Matej Artac², Tadej Borovšak², Elisabetta Di Nitto¹, Michele Guerriero¹, Damian Andrew Tamburri¹

¹DEEPSE group - DEIB - Politecnico di Milano, Milano, Italy

²XLAB - Ljubljana, Slovenia

matej.artac@xlab.si, tadej.borovsak@xlab.si, elisabetta.dinitto@polimi.it,

michele.guerriero@polimi.it, damianandrew.tamburri@polimi.it

Abstract—DevOps entails a series of software engineering tactics aimed at shortening the actionable operation of software design changes. One of these tactics is to harness infrastructure-as-code, that is, writing a blueprint that contains deployment specifications ready for orchestration in the cloud. This abstract briefly discusses all necessary elements and abstractions in writing and maintaining that blueprint, revolving around a key standard for its expression, namely, the OASIS “Topology and Orchestration Specification for Cloud Applications” (TOSCA), industrial standard adopted by as many as 60+ big industrial players worldwide.

Keywords—DevOps; Infrastructure-as-Code; TOSCA;

I. INTRODUCTION

The current IT market is increasingly dominated by the “need for speed”. This need is reflected in the emerging use of agile and lean techniques which shorten the software development cycle and also intermix software development activities with IT operations [1]. This trend of using software engineering tactics that reduce the space, time and efforts between software development and operations as well as the technical and organizational distance between these two types of software teams is known as DevOps [1]. As part of the DevOps menu, many practices entail re-using standard tools from software development (e.g., code-versioning, code-revision management, etc.) to manage what is known as *infrastructure-as-code* (IaC) [2]. IaC promotes managing knowledge and experience of plethora of subsystems as a single commonly available source of truth instead of traditionally reserving it for system administrators. This briefing introduces all essential technologies involved in supporting infrastructure-as-code, starting from the industrial standard for IaC, OASIS TOSCA, which stands for “Topology and Orchestration Specification for Cloud Applications” [3] and elaborating more on connected technologies, such as deployment blueprints or TOSCA-ready orchestration engines. TOSCA is state-of-the-art industrial practice language for automated deployment of technology independent and multi-cloud compliant applications, supported by players such as HP, CISCO Systems, Cloudera, Huawei, IBM [3].

The following contains a brief overview of DevOps, addressing TOSCA and IaC together.

II. DEVOPS IN PILLS

In essence the core idea behind DevOps is fairly simple: over long years of “siloed” [4] software development (on

one side) and software operations (on the other), a series of distances in technical, socio-technical, and organizational practices and tools has been flourishing between Dev- and -Ops teams [1]. These distances make the fluent and coherent communication, collaboration and issue resolution between the two silos practically difficult if not impossible. DevOps entails a series of socio-technical and organizational practices and tools that address and tackle these distances one by one until a *frictionless*, fluent, and resilient organization is distilled. Of the several socio-technical practices under study we mention, most prominently, the organizational blending of Dev- and -Ops teams under the same working unit with consequent shifting of operational concerns (think of, for example, infrastructure design, dimensioning, cost-optimization [5]) to be addressed by the entire team, including developers. Other similar socio-technical and organizational practices lean towards the shared use of “DevOps” tools [6], i.e., tools that promote the use of the same knowledge-base (think of an issue-tracking mechanism) for both development and operation such that there shall be no misunderstanding or knowledge gap between developer and operator knowledge at all.

III. INTRODUCING INFRASTRUCTURE-AS-CODE

Infrastructure design is the software lifecycle phase that defines and configures the software infrastructure needs for that software, e.g., the AWS cloud, the number and type of VMs required, etc. Infrastructure designs typically entail many installation and configuration scripts needed to, among others: (a) instantiate and link the required machines (either physical or virtual) for the software to run; (b) install and configure the required software and middleware for those VMs; (c) instantiate and run the needed ancillary services for the software to be operated. In the scope of these designs, DevOps promotes the use of a typical software development notion, i.e., “source code”, for infrastructure designs as well, such that the entire set of scripts, automation and configuration code, models, required dependencies and operational configuration parameters can be expressed using the same standard language, much like you would use Java and ancillary Java libraries to put together your own software application - this practice is *infrastructure-as-code*. The purpose behind this proposal is to re-use successful and common software development practices to speed up software operations: think of using infrastructure code versioning for the purpose of de-bugging and back-tracking/version control or using infrastructure design

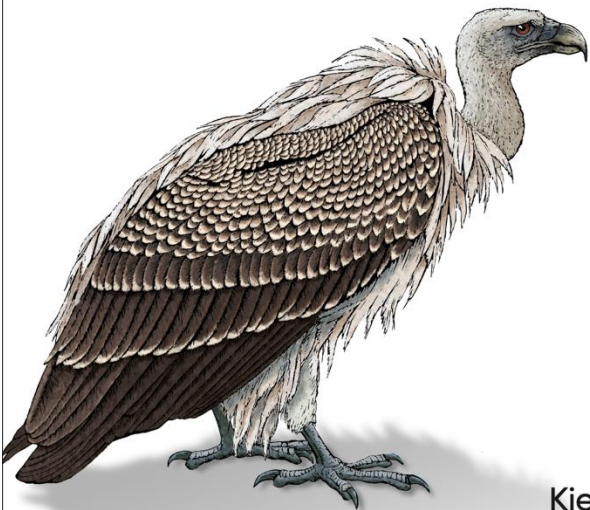


O'REILLY®

Infrastructure as Code

Dynamic Systems for the Cloud Age

Second Edition

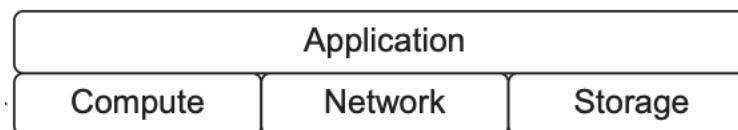


Kief Morris

What is Infrastructure as Code

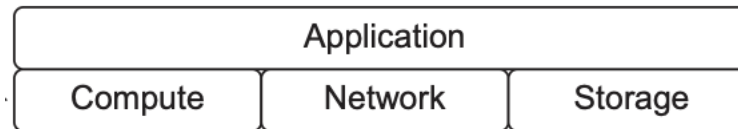
- How does IaC work?
- What problem does it solve?
- What practices and patterns can be applied to writing IaC?

What is Infrastructure



The data centre definition of infrastructure includes networking, compute, and storage resources used to run an application

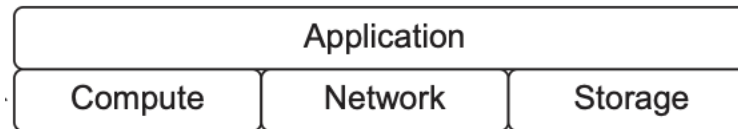
What is Infrastructure



Is the following checklist sufficient to satisfy a request to help deliver an application to production for a user?

- Enough servers ✓
- Network connectivity for users ✓
- A database for storing application data ✓

What is Infrastructure

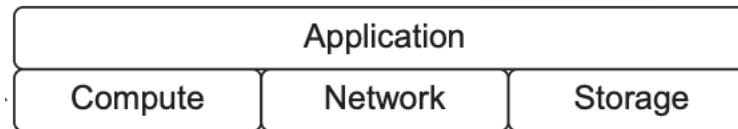


This definition of infrastructure omits some critical tasks necessary for production readiness, including:

- Monitoring application metrics
- Adding health checks for servers and databases
- Supporting user authentication
- Logging and aggregating application events
- etc

Infrastructure-related
Operations

What is Infrastructure

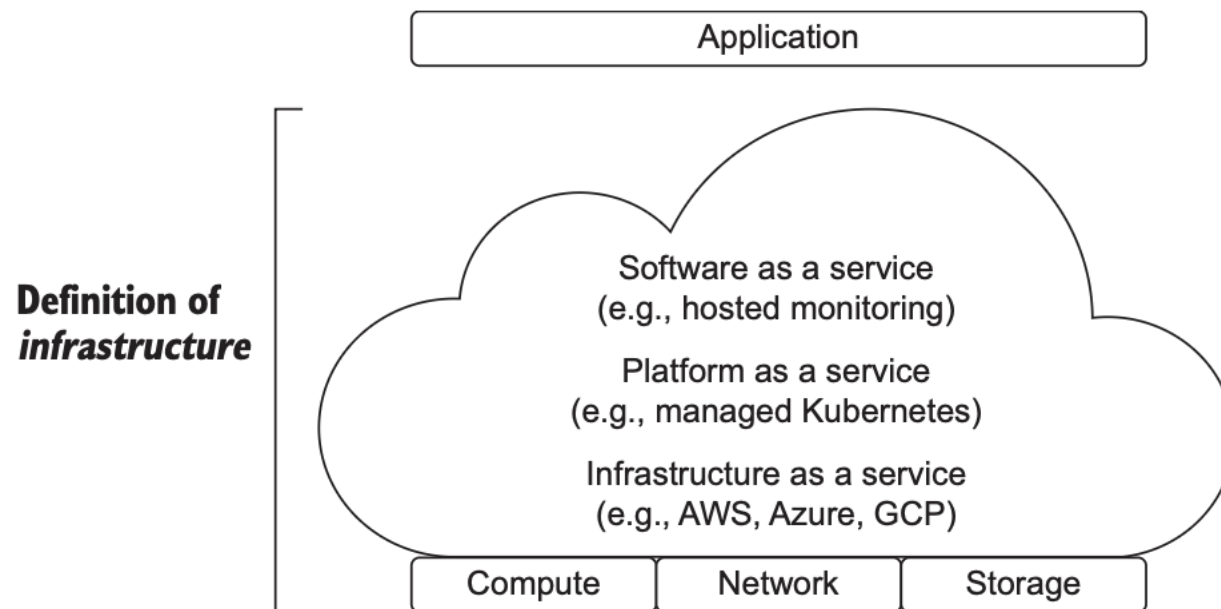


This definition of infrastructure omits some critical tasks necessary for production readiness, including:

- Platform as a Service (PaaS)
- Software as a Service (SaaS)
- Function as a Service (or Container as a Service)
- Anything as a Service (XaaS)

Different abstractions for the management of base networking, compute, and storage

What is Infrastructure



The infrastructure for an application might include queues on a public cloud, containers running applications, serverless functions for additional processing, or even monitoring services to check the system's health.

What is Infrastructure

DEFINITION

Infrastructure refers to the software, platform, or hardware that delivers or deploys applications to production.

Here is a non-exhaustive list of infrastructure you might encounter:

- Servers
- Workload orchestration platforms
(e.g., Terraform, Ansible, Kubernetes,
HashiCorp Nomad, Docker,
Docker Swarm, etc)
- Network switches
- Load balancers
- Databases
- Object stores
- Caches
- Queues
- Event-streaming platforms
- Monitoring platforms
- Data pipeline systems
- Payment platforms

Class Discussion

(1) What is the biggest risk to a production system?

Optimising for Change

Changes cause 80% of unplanned outages

Gene Kim, George Spafford, and Kevin Behr in *The Visible Ops Handbook* (IT Process Institute)

- Continuous change is inevitable, and
- Making changes is the only way to improve a system

It therefore makes sense to optimise the capability of your infrastructure development process to make changes both rapidly and reliably.

Common examples of Infrastructure Changes

- An essential new application feature requires you to add a new database.
- A new application feature needs you to upgrade the application server.
- Usage levels grow faster than expected. You need more servers, new clusters, and expanded network and storage capacity.
- Performance profiling shows that the current application deployment architecture is limiting performance. You need to redeploy the applications across different application servers. **Doing this requires changes to the clustering and network architecture.**

Common examples of Infrastructure Changes

- There is a newly announced security vulnerability in system packages for your OS. You need to patch dozens of production servers.
- You need to update servers running a deprecated version of the OS and critical packages.
- Your web servers experience intermittent failures. You need to make a series of configuration changes to diagnose the problem. Then you need to update a module to resolve the issue.
- You find a configuration change that improves the performance of your database.

Optimising for Change

A fundamental truth of the Cloud Age is: **Stability** comes from making changes.

- Unpatched systems are not stable, they are vulnerable
- If you can't fix issues as soon as you discover them, your system is not stable.
- If you can't recover from failure quickly, your system is not stable.
- If the changes you do make involve considerable downtime, your system is not stable.
- If changes frequently fail, your system is not stable.

Technical debt

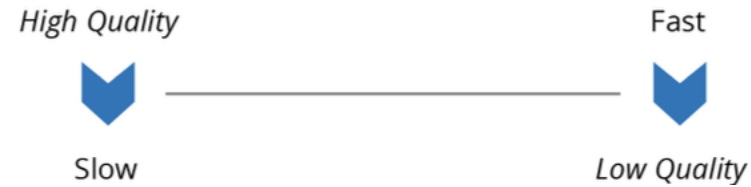
DEFINITION

Technical debt refers to the implied cost of additional rework caused by choosing an easy or limited solution now instead of using a better approach that would take longer.

- Outdated technologies or legacy code
- Rushed development to meet deadlines
- Poor documentation or lack of standards
- Inadequate testing
- Quick fixes or “temporary” solutions that become permanent

Class Discussion

(2) To engineer better systems, we must choose between speed and quality?



The Quadrant Model

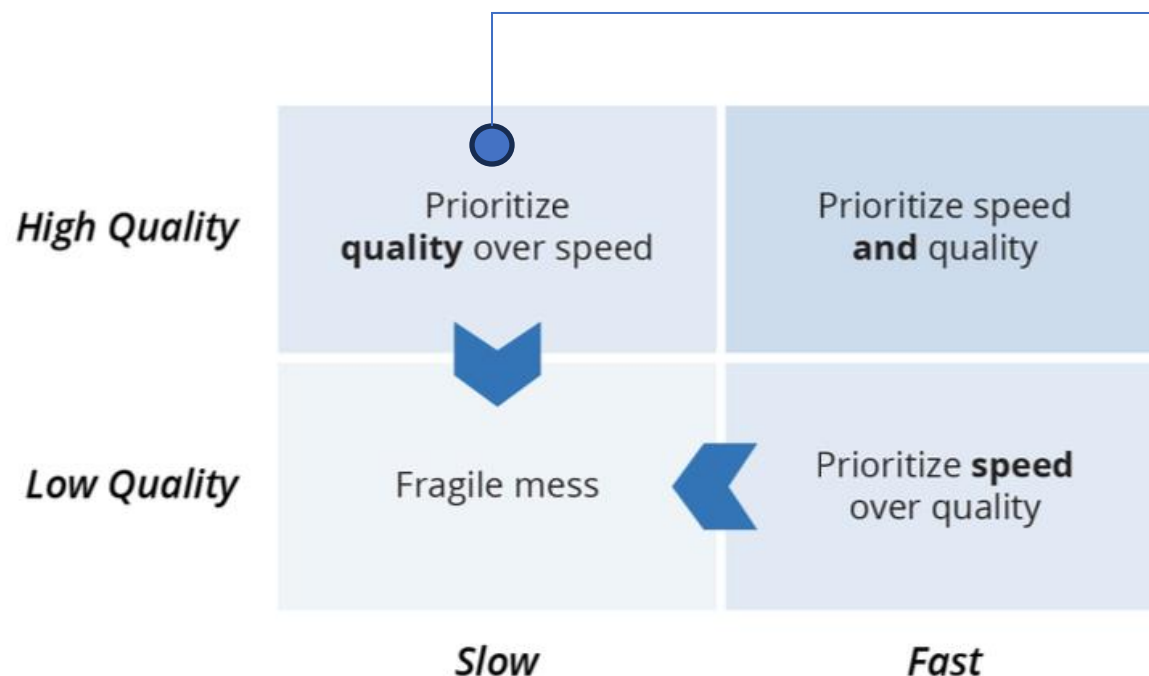


Prioritize speed over quality:

“move fast and break things” philosophy.

- Teams that optimize for speed and sacrifice quality build messy, fragile systems.
- They slide into the lower-left quadrant because their shoddy systems slow them down.

The Quadrant Model

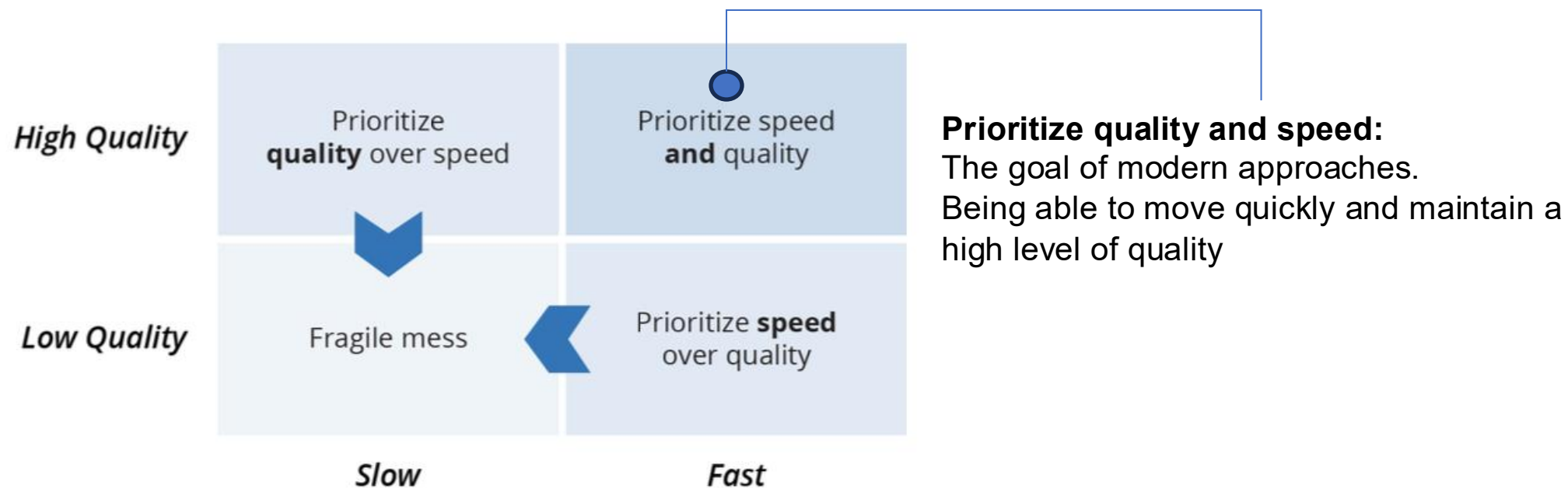


Prioritize quality over speed:

Also known as, “We’re doing serious and important things, so we have to do things properly.”

- Heavyweight processes create barriers to improvement, so technical debt grows along with lists of “known issues.”
- These teams slump into the lower-left quadrant.
- They add more processes in response to failures.
- These processes make it even harder to make improvements and increases fragility and risk.

The Quadrant Model



Manual configuration of infrastructure

Setting up virtual networks, servers, storage, and security settings directly through consoles or configuration files (bash scripts)—without automation tools.

- No version control: Changes aren't tracked unless documented separately.
- Hard to scale: Not ideal for large or dynamic environments
- No testing: Commands are executed without testing their effects or checking their intent.
- Error-prone: Easy to misconfigure settings, difficult to detect root cause, with no idea of which resources a change would affect (blast radius)

Manual configuration of infrastructure

DEFINITION

The blast radius refers to the impact a failed change has on a system. A larger blast radius often affects more components or the most critical components

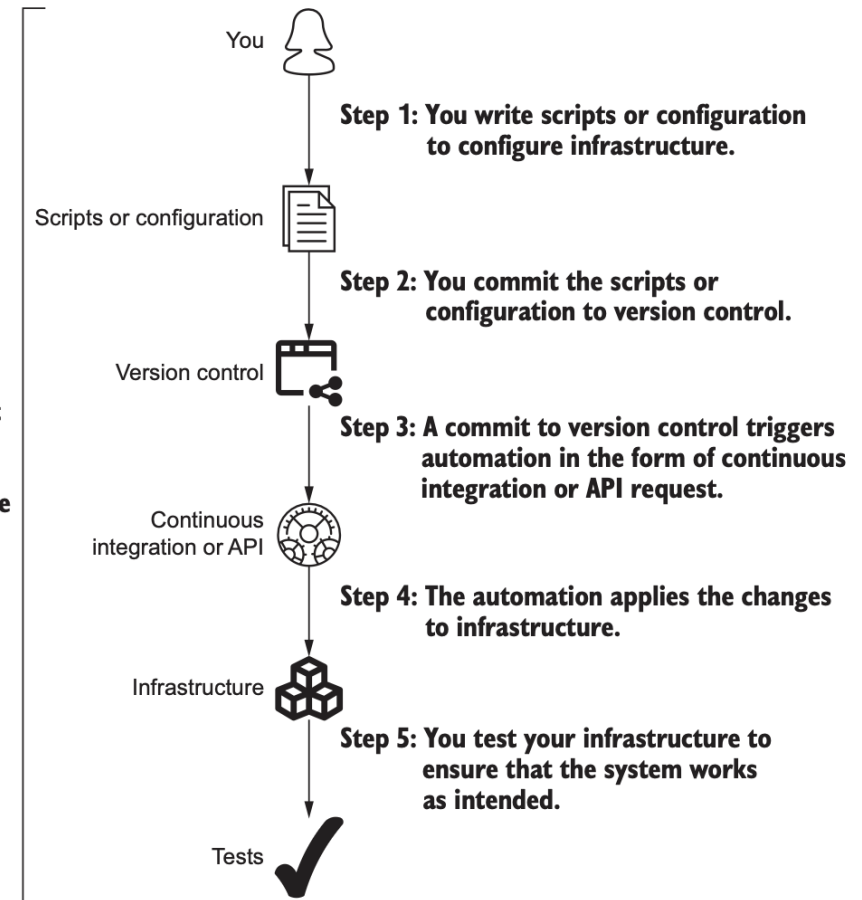
- Manual changes increase the change failure rate of systems, slow development, and expose the system's attack surface to a potential security exploit.

Infrastructure as Code

What should you do to change infrastructure, if not manual changes?

- Apply a software development life cycle for infrastructure resources and configuration in the form of IaC.
- A development life cycle for infrastructure includes writing the code as documentation, committing it to version control, applying it to infrastructure in an automated way, and testing it.

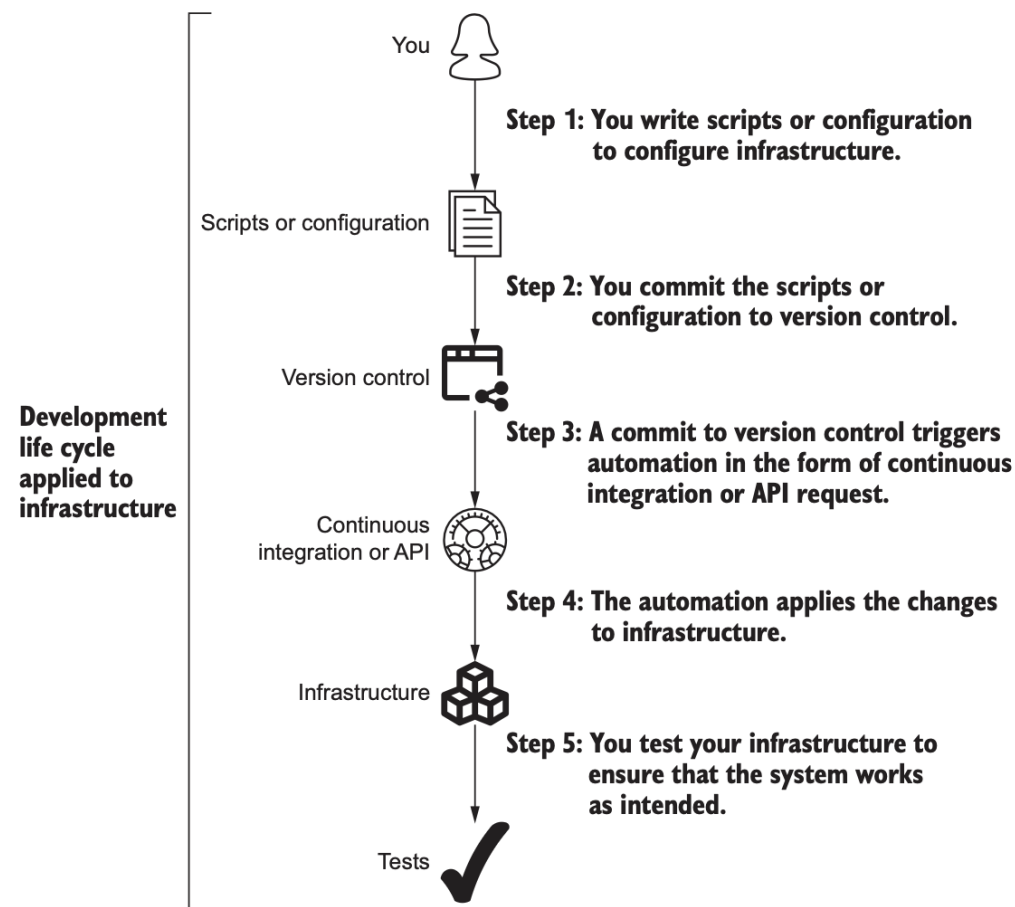
Development
life cycle
applied to
infrastructure



The Development Lifecycle

A **general pattern** for managing changes and verifying that they don't negatively affect the system.

- This lifecycle captures infrastructure as code, which automates infrastructure changes in a codified manner.
- Applies DevOps practices such as version control and continuous delivery

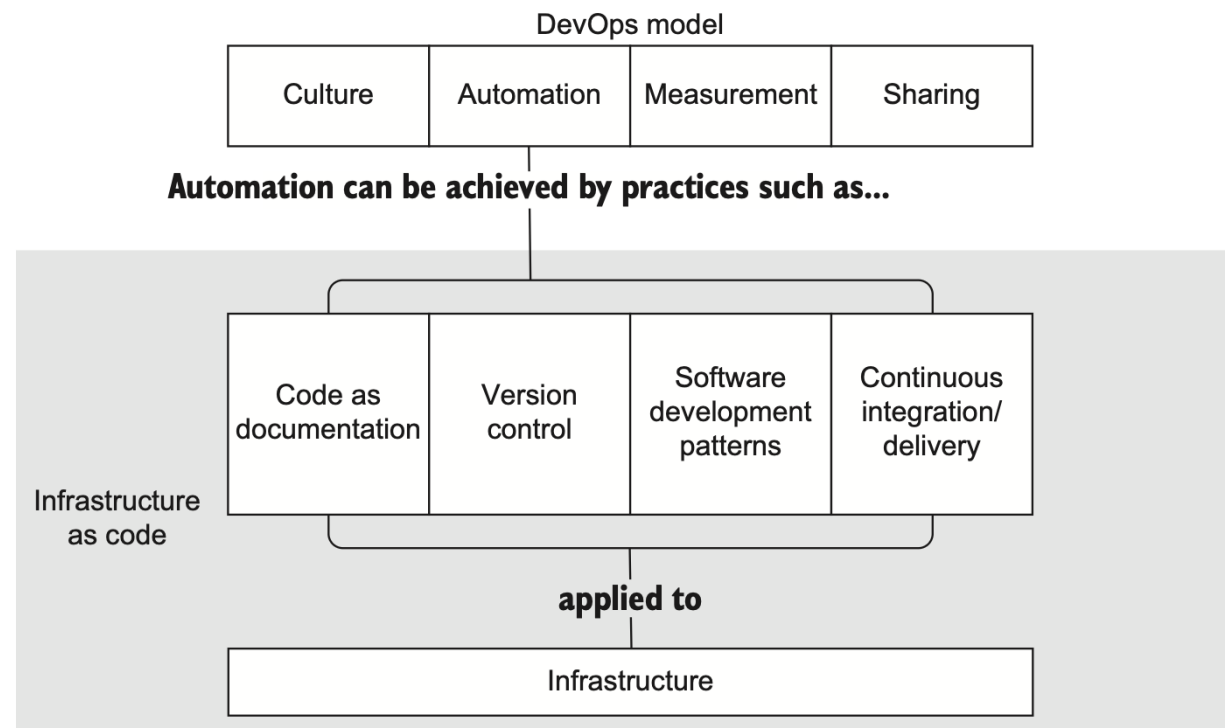


The Development Lifecycle

DEFINITION

Infrastructure as code (IaC) applies DevOps practices to automating infrastructure changes in a codified manner to achieve scalability, resiliency, and security

- Infrastructure as code applies DevOps practices to automating changes in a codified manner. These practices include version control and continuous integration or delivery



Infrastructure as Code

DEFINITION

Infrastructure as Code is an approach to infrastructure **automation** based on practices from software development. It emphasises **consistent, repeatable** routines for provisioning and changing systems and their configuration.

You make changes to code, then use automation to test and apply those changes to your systems.

- Agile engineering practices
- Test Driven Development (TDD)
- Continuous Integration (CI)
- Continuous Delivery (CD)

Principles of infrastructure as code

- Four most important principles (*mnemonic RICE*):

- *Reproducibility*

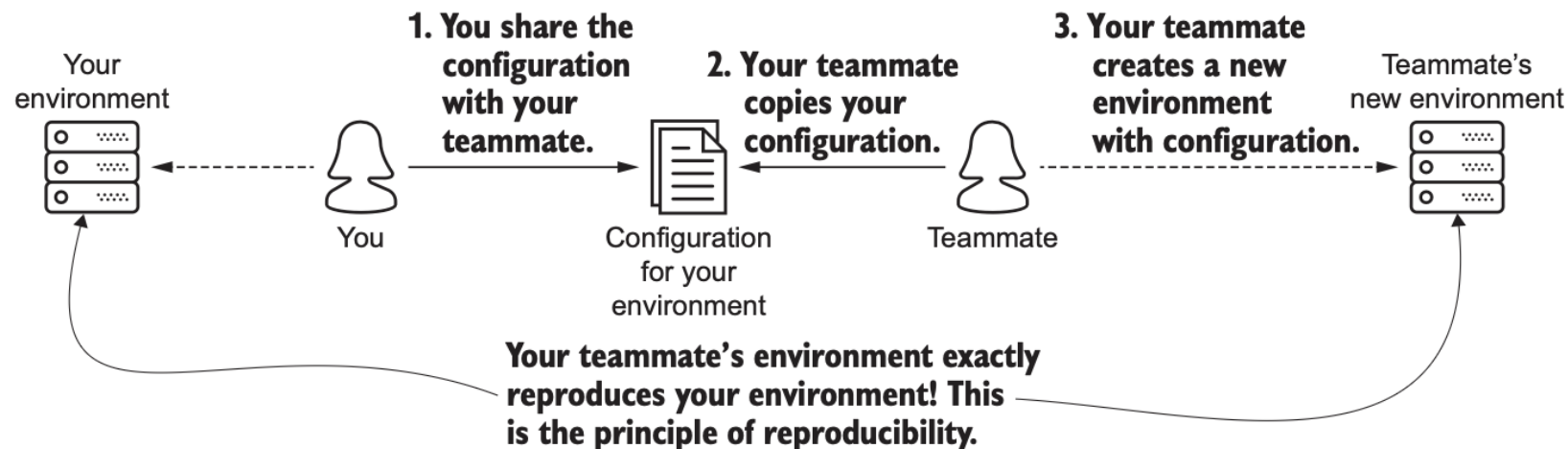
- *Idempotency*

- *Composability*, and

- *Evolvability*

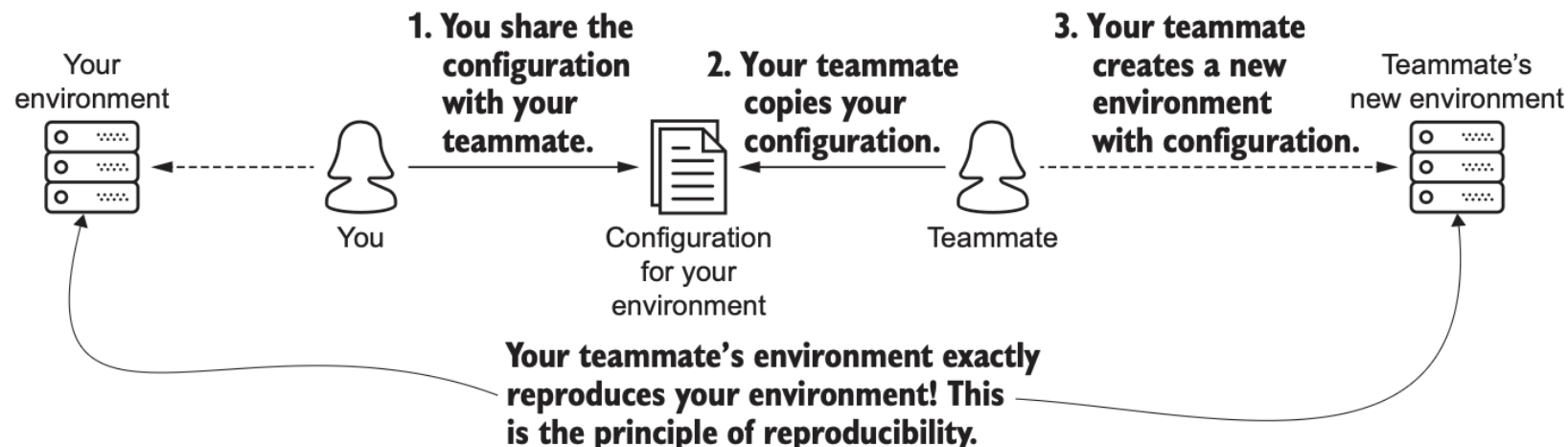
The RICE framework

Reproducibility



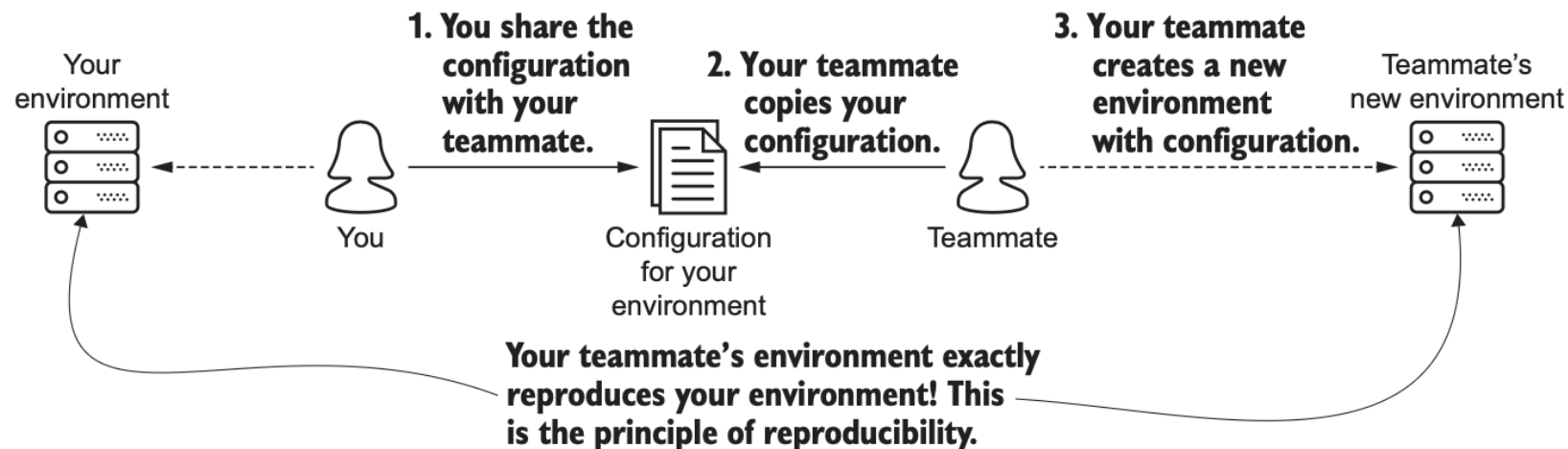
- The ability to copy and reuse infrastructure configuration (You don't have to reinvent the wheel to create new environments or infrastructure resources)
- Manual changes introduce **drift** between version control and the actual state and affect reproducibility, so you instead update changes in version control.

Reproducibility



- The ability to copy and reuse infrastructure configuration (You don't have to reinvent the wheel to create new environments or infrastructure resources)
- Manual changes introduce **drift** between version control and the actual state and affect reproducibility, so you instead update changes in version control.

Reproducibility



DEFINITION

The principle of reproducibility means that you can use the same configuration to reproduce an environment or infrastructure resources.

Reproducibility

Scenario

Imagine you need to reduce a network address space from /16 to /24. You do have IaC written that expresses the network. However, you decide to log into the cloud provider and type /24 into the text box.

Reflect on whether your change workflow adheres to reproducibility. You ask yourself the following questions:

- Will a teammate know that you've updated the network?
- If you run your configuration, will the network address space return to the /16?
- If you create a new environment with the configuration in version control, will it have an address space of /24?

Reproducibility

Scenario

Imagine you need to reduce a network address space from /16 to /24. You do have IaC written that expresses the network. However, you decide to log into the cloud provider and type /24 into the text box.

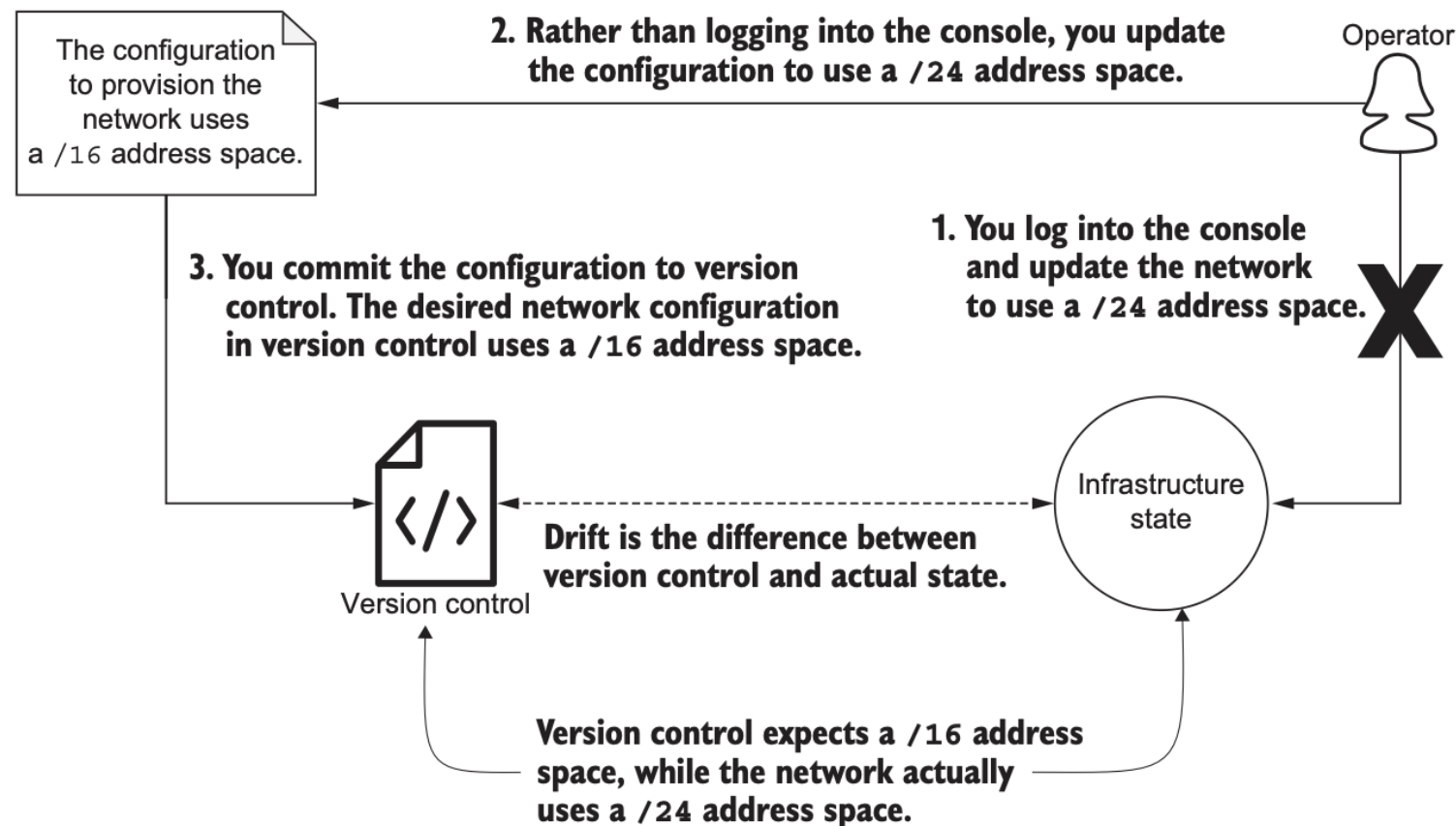
Reflect on whether your change workflow adheres to reproducibility. You ask yourself the following questions:

- Will a teammate know that you've updated the network?
- If you run your configuration, will the network address space return to the /16?
- If you create a new environment with the configuration in version control, will it have an address space of /24?

If you answer no to any of these questions, you cannot guarantee that you will reproduce the manual change successfully

Reproducibility

- Manual changes introduce drift between version control and the actual state. This affects reproducibility, so you instead update changes in version control.



Reproducibility

Other common causes of configuration drift:

- **Untracked Updates.** These are updates made without version control or proper documentation.
- Using **multiple IaC Tools or Scripts** (e.g., Terraform and CloudFormation) for overlapping resources can cause conflicts. Scripts may overwrite each other's configurations.
- External scripts or CI/CD pipelines that modify infrastructure without updating IaC definitions. This is common in legacy systems or hybrid setups
- Lack of Governance or Access Controls resulting in too many users with permissions to change infrastructure directly. Without guardrails, ad hoc changes become frequent

Reproducibility

DEFINITION

Configuration drift is the deviation of infrastructure configuration from the desired configuration to the actual configuration

How to Prevent Configuration Drift

- Enforce IaC-only changes to restrict direct access to cloud consoles.
- Use drift detection tools like Terraform's terraform plan, AWS Config, or Driftctl.
- Implement CI/CD pipelines by automating deployments and ensuring changes go through code reviews.
- Audit and monitor regularly: Track changes and reconcile discrepancies.

Idempotency

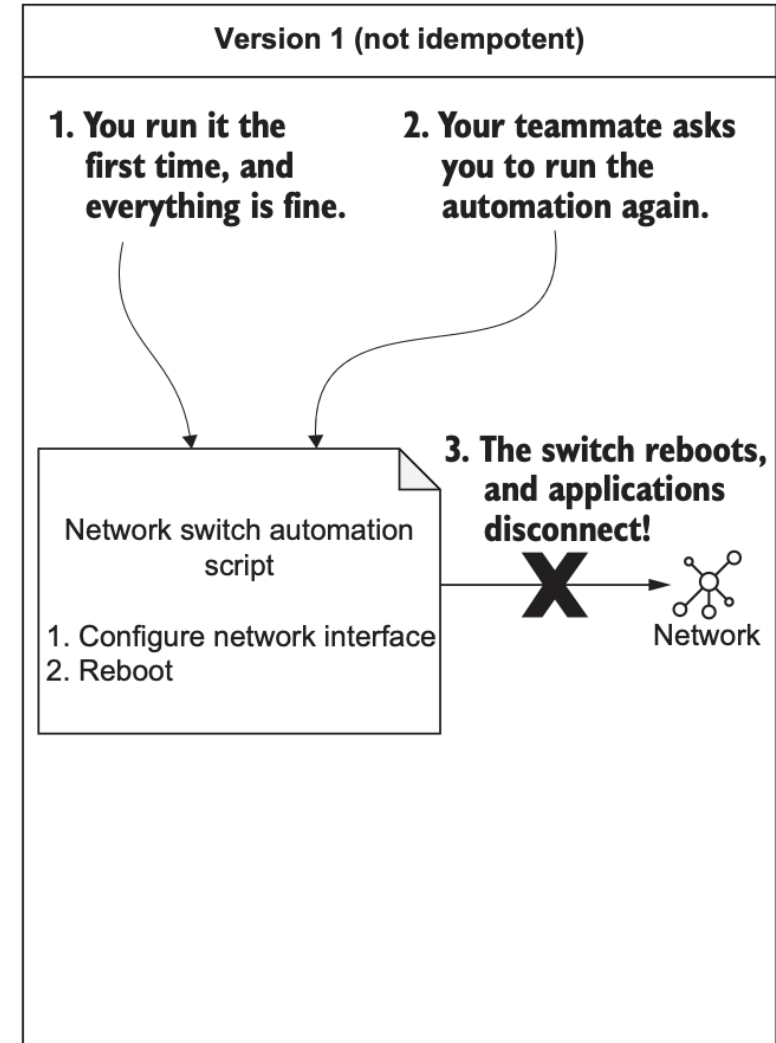
Running automation should result in the same end state for an infrastructure resource. This ensures:

- Consistency (Repeated deployments don't introduce unintended changes)
- Predictability (You can safely reapply configurations without worrying about breaking things)
- Automation-friendly (Makes CI/CD pipelines and automated provisioning reliable)

Idempotency

Scenario (1)

Imagine you write a network script that first configures an interface and then reboots. The first time you run the script, the switch configures the interface and reboots. You save this script as version 1. A few months later, your teammate asks you to run the script again on the switch. You run the script, and the switch reboots. However, the reboot disconnects some critical applications!



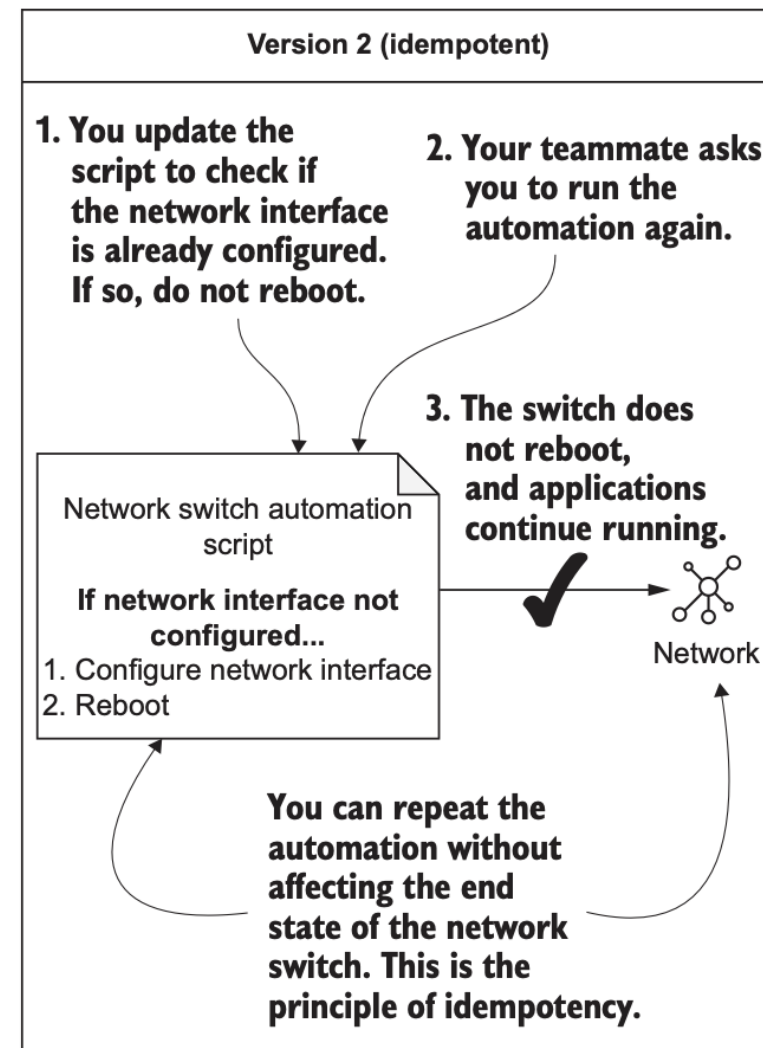
Idempotency

Scenario (2)

You figure out a way to prevent the switch reboot if you've already configured the network interface. You create version 2 of the script and add a conditional if statement.

The statement checks whether you have configured the interface already before rebooting the switch. When you run the version 2 script again, you do not disconnect the applications.

The conditional statement conforms to the principle of idempotency.



Idempotency

Code Example Terraform script that creates an AWS EC2 instance:

```
resource "aws_instance" "web" {  
    ami          = "ami-123456"  
    instance_type = "t2.micro"  
}
```

- The **first time** you run the command **terraform apply**, it creates the instance.
- The **second time**, Terraform sees that the instance already exists and **does nothing**—unless the configuration has changed.
- This is idempotency: no change unless necessary.

Idempotent vs Non-Idempotent

Action	Idempotent?	Why?
Creating a VM with same config	Yes	No change if already exists
Rebooting a VM	No	Each run triggers a reboot
Setting a firewall rule	Yes	Rule is added only once

Idempotent Operations:

- Creating a resource that already exists – the tool checks and skips creation.
- Updating a resource to the same configuration - no change is made.
- Deleting a non-existent resource – No error is thrown and the system remains unchanged

Idempotency

DEFINITION

The principle of idempotency ensures that you can repeatedly run the automation on infrastructure without affecting its end state or having any side effects. You should affect infrastructure only when you update its attributes in automation.

Composability

Goal:

- To mix and match any set of infrastructure components, no matter the tool or configuration.
- To update the individual configurations without affecting the entire system

Both requirements promote modularity and decoupling of infrastructure dependencies

Composability

Scenario

Imagine you create infrastructure for an application that someone accesses at www.hello-world.com. What are the minimum resources you need for a secure and production-ready configuration?

- A server
- A load balancer
- A private network for a server
- A public network for a load balancer
- A routing rule to allow traffic out of the private network
- A routing rule to allow public traffic to the load balancer
- A routing rule to allow traffic from the load balancer to the server
- A domain name for www.hello-world.com

Implementation Options:

(1) Write this configuration from scratch,

or

(2) Look for preconstructed modules that group infrastructure components you can use to assemble the system

Terraform Registry

The official repository for Terraform modules. Host a wide range of reusable modules for cloud providers like AWS, Azure, Google Cloud, Proxmox, Kubernetes, etc

<https://registry.terraform.io>

Contains:

- Verified modules that are maintained by HashiCorp and trusted partners.
- Community modules that are shared by users and organisations.
- Searchable by providers
- Versioned and documented. Each module includes usage examples, inputs, outputs, and dependencies.

Composability in Terraform

Example - Reusing a tagging module:

You want to apply consistent tags to all your AWS resources. Instead of repeating tag blocks everywhere, you create a small module.

modules/tags/outputs.tf

```
output "common_tags" {  
  value = {  
    Environment = var.environment  
    Owner       = var.owner  
  }  
}
```

modules/tags/variables.tf

```
variable "environment" {}  
variable "owner" {}
```

Composability in Terraform

Example - Reusing a tagging module:

You want to apply consistent tags to all your AWS resources. Instead of repeating tag blocks everywhere, you create a small module.

Main.tf

```
module "tags" {  
    source      = "../modules/tags"  
    environment = "dev"  
    owner       = "team-a"  
}  
  
resource "aws_s3_bucket" "example" {  
    bucket = "my-example-bucket"  
    tags   = module.tags.common_tags  
}
```

terraform.tfvars

```
# terraform.tfvars  
environment = "dev"  
owner       = "team-a"  
instance_type = "t2.micro"
```

Reuse in root configuration

Composability in Terraform

Example - Reusing a tagging module:

You want to apply consistent tags to all your AWS resources. Instead of repeating tag blocks everywhere, you create a small module.

- Keeps tagging logic centralised.
- Makes it easy to update tags across resources.
- Promotes consistency and reuse with minimal code.

Composability

Scenario

Imagine you create infrastructure for an application that someone accesses at www.hello-world.com. What are the minimum resources you need for a secure and production-ready configuration?

- A server
- A load balancer
- A private network for a server
- A public network for a load balancer
- A routing rule to allow traffic out of the private network
- A routing rule to allow public traffic to the load balancer
- A routing rule to allow traffic from the load balancer to the server
- A domain name for www.hello-world.com

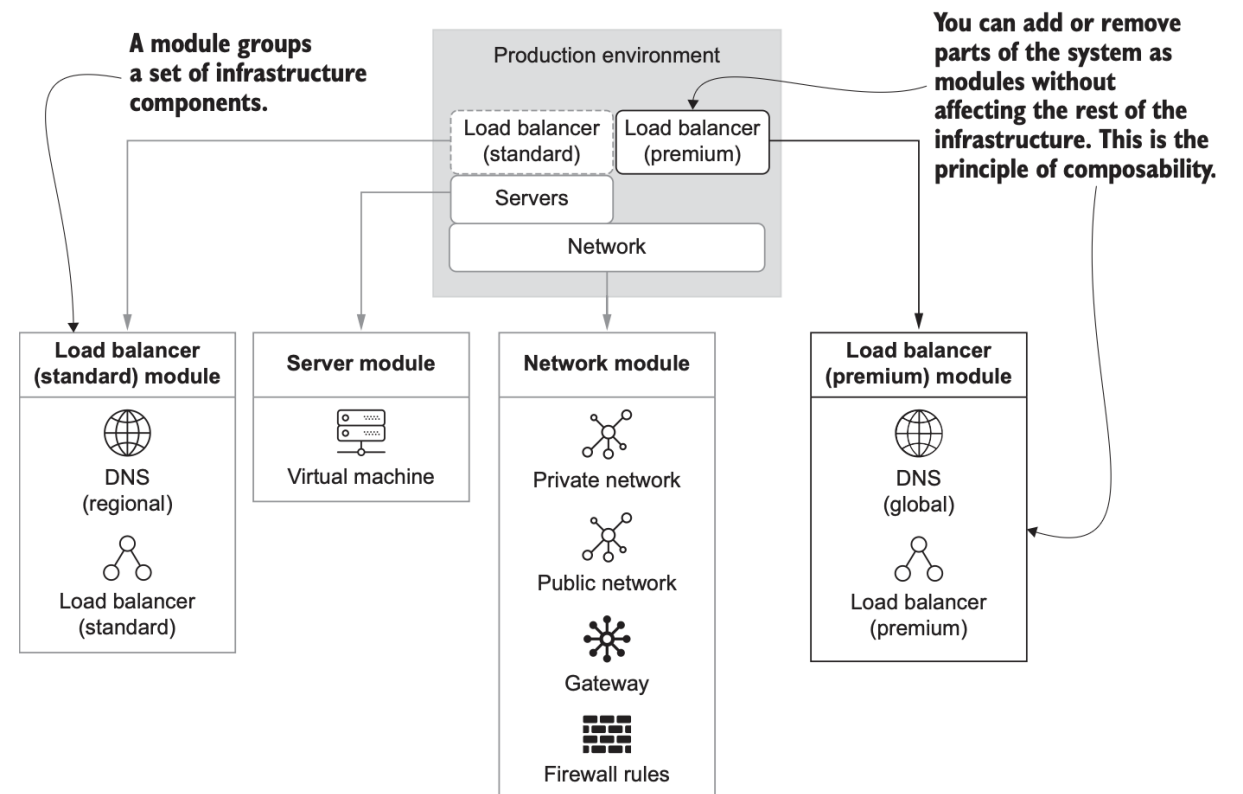
You now have multiple modules that create the following:

- Networks (private and public networks, gateways to route traffic from the private network, routing rules to allow traffic out of the private network)
- Servers
- Load balancers (domain name, routing rules to allow traffic from the load balancer to the server)

Composability

You build your production environment with building blocks of infrastructure so you can add new resources, like the premium load balancer, easily

- A server
- A load balancer
- A private network for a server
- A public network for a load balancer
- A routing rule to allow traffic out of the private network
- A routing rule to allow public traffic to the load balancer
- A routing rule to allow traffic from the load balancer to the server
- A domain name for www.hello-world.com



Composability

DEFINITION

Composability ensures that you can assemble any combination of infrastructure resources and update each one without affecting the others.

- The more composable your configuration, the easier it is to create new systems with less effort.
- The lack of composability of IaC runs the risk of change failures due to unknown dependencies in complex infrastructure systems.

Evolvability

Goal:

- You want to account for the scale and growth of your system, but not optimise the configuration too early and unnecessarily

Evolvability in practice

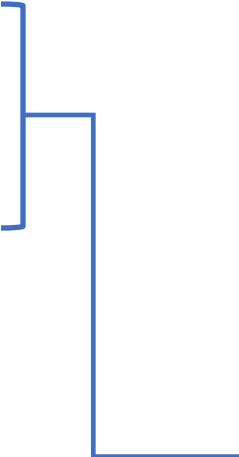
- Ease of change: You can modify infrastructure definitions without rewriting everything.
- Backward compatibility: Updates don't disrupt existing deployments.
- Scalability: Infrastructure can grow in complexity without becoming unmanageable.
- Maintainability: Code remains readable, testable, and modular over time.

Evolvability in Terraform

Example – Evolving VPC module

Main.tf

```
module "vpc" {  
    source = "../modules/vpc"  
    cidr   = "10.0.0.0/16"  
}
```



stating with basic functionality.

Evolvability in Terraform

Example – Evolving VPC module

Main.tf

```
module "vpc" {  
  source = "../modules/vpc"  
  cidr    = "10.0.0.0/16"  
  name    = "my-vpc"  
  
  azs                = ["us-west-2a", "us-west-2b"]  
  public_subnets    = ["10.0.1.0/24", "10.0.2.0/24"]  
  private_subnets   = ["10.0.101.0/24", "10.0.102.0/24"]  
  
  enable_flow_log = true  
  create_flow_log_cloudwatch_iam_role = true  
  create_flow_log_cloudwatch_log_group = true  
}
```

Later, evolve it to support:

- Multiple availability zones
- Public/private subnets, and
- Logging

Without changing how the module is consumed—just by updating its internals and inputs.

Evolvability

DEFINITION

The principle of evolvability ensures that you can change your infrastructure resources to accommodate system scale or growth while minimising effort and risk of failure.

- The ability to adapt, extend, and improve infrastructure definitions over time without breaking existing systems.
- It ensures that your IaC codebase can grow with your organisation's needs while remaining maintainable and reliable.

Applying Principles

As you write IaC, ask whether you conform to all four principles. As you write IaC, ask whether you conform to all four principles.

- These principles help you write and share your IaC with less effort and ideally minimize the impact of changes to your system.
- A missing principle can hinder updates to infrastructure resources or increase the blast radius of potential failure.

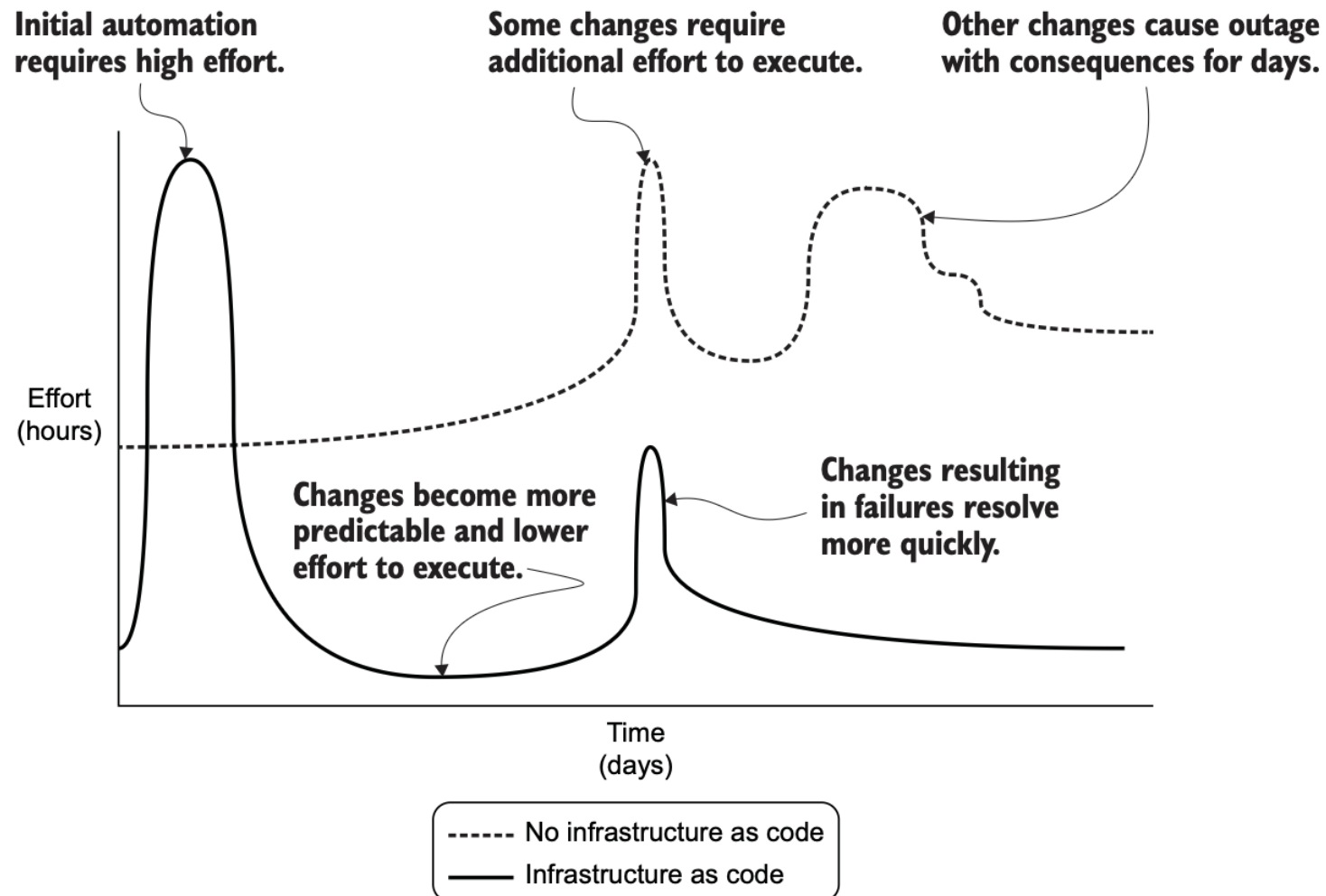
Applying Principles

As you write IaC, ask whether you conform to all four principles. As you write IaC, ask whether you conform to all four principles.

Reproducibility	Idempotency	Composability	Evolvability
Can you use the same configuration to reproduce an environment or infrastructure resources? Development Environment Configuration You can copy the configuration to create a matching set of infrastructure resources. Production Environment Configuration	Can you repeatedly run the automation on infrastructure without changing its current state? I run ... □ Step 1 □ Step 2 □ Step 3 They result in the same end state. Given I have run step 1, I run ... □ Step 2 □ Step 3	Can you assemble any combination of infrastructure resources without building a new system? You can add a cache to the system easily. Container orchestrator Virtual machine Network Cache	Can you change your infrastructure and minimize disruption of the overall system? You can swap a database and minimize disruption to your applications. Applications Relational database Network NoSQL database

Why use infrastructure as code?

- IaC requires less effort over time after the high initial automation effort. Without IaC, time to execute changes can be highly variable.



IaC Platforms/Tools

- IaC tooling varies quite widely because it applies to various resources.
- Most tools and platforms fall into one of three use cases, all of which address very different functions and vary widely in behaviour, including the following:
 1. Provisioning
 2. Configuration management
 3. Image building

Provisioning

- Provisioning tools create and manage sets of infrastructure resources for a given provider, whether it's a public cloud, data centre, or hosted monitoring solution.
- A provider refers to a data centre, IaaS, PaaS, or SaaS responsible for providing infrastructure resources.

Tool	Provider
AWS CloudFormation	Amazon Web Services
Google Cloud Deployment Manager	Google Cloud Platform
Azure Resource Manager	Microsoft Azure
Bicep	Microsoft Azure
HashiCorp Terraform	Various (declarative)
Pulumi	Various (mostly imperative)
AWS Cloud Development Kit	Amazon Web Services
Kubernetes manifests	Kubernetes (container orchestrator)

Provisioning – Dry Run

- Most provisioning tools can preview changes to a system and express dependencies between infrastructure resources, also known as a *dry run*.

DEFINITION

A *dry run* analyses and outputs expected changes to infrastructure before you apply the changes to resources.

For example, you can express a dependency between a network and a server. If you change the network, the provisioning tool will show that the server may also change.

Configuration management

- Configuration management tools ensure that servers and computer systems run in the desired state

DEFINITION

A configuration management tool configures a set of servers or resources for the packages and attributes hosted on them.

For example, if you have 10,000 servers in your data centre, how do you ensure that all of them run a specific version of the packages your security team approved?

Configuration management

■ Non-exhaustive list of configuration management tools

Tool	Type	Key Features	Best For
Ansible	Agentless	YAML playbooks, SSH-based, simple syntax	Cloud-native automation
Puppet	Agent-based	Declarative language, centralized control, strong reporting	Large-scale enterprise environments
Chef	Agent-based	Ruby DSL, flexible configuration, test-driven development	Complex infrastructure customization
SaltStack	Agent/Agentless	Event-driven automation, fast remote execution	Real-time orchestration
CFEngine	Agent-based	Lightweight, scalable, policy enforcement	High-performance, legacy systems

Configuration management

■ Non-exhaustive list of configuration management tools (contd)

Tool	Type	Key Features	Best For
Rudder	Agent-based	Web UI, compliance reporting, policy-driven automation	Regulated industries, audit trails
Otter	Agentless	Visual orchestration, drift detection, PowerShell support	Windows-heavy environments
Fabric	Agentless	Python-based scripting, SSH execution	Lightweight automation via Python
Foreman	Agent-based	Lifecycle management, provisioning, integration with Puppet	Bare-metal and VM provisioning
Auvik	Agentless	Network monitoring, configuration backup, performance analytics	Network infrastructure visibility
ManageEngine Endpoint Central	Agent-based	Patch management, remote control, software deployment, multi-platform support	Endpoint security and compliance

■ This unit focuses on provisioning tools and managing singleprovider systems

Image Building

- When you create a server, you must specify a machine image with an operating system
- Image-building tools create images used for application runtime, whether a container or server

DEFINITION

An image-building tool builds machine images for application run times, such as containers or server

Tool	Runtime Environment	Build target
HashiCorp Packer	Containers and servers	Various
Docker	Containers	Container registry
EC2 Image Builder	Servers	Amazon Web Services
Azure VM Image Builder	Servers	Event-driven automation, fast remote execution
CFEngine	Agent-based	Microsoft Azure

IaC Visualisation

Tools for Infrastructure diagramming

- Lucichart: Web-based diagramming tool with AWS, Azure, GCP icons (<https://www.lucidchart.com>)
- Draw.io (diagrams.net): Free, browser-based tool with cloud templates
- Structurizr: Code based modelling
- PlantUML: Text-based UML and architecture diagrams
- Microsoft Visio: Professional diagramming software with rich templates
- Cloudcraft: AWS-specific visual designer with cost estimation
- Terrastruct: Designed for layered architecture diagrams

Benefits of Infrastructure as Code

Organisations adopting Infrastructure as Code to manage dynamic infrastructure hope to achieve benefits, including:

- Using IT infrastructure as an enabler for rapid delivery of value
- Reducing the effort and risk of making changes to infrastructure
- Enabling users of infrastructure to get the resources they need, when they need it
- Providing common tooling across development, operations, and other stakeholders
- Creating systems that are reliable, secure, and cost-effective
- Make governance, security, and compliance controls visible
- Improving the speed to troubleshoot and resolve failures

Summary

- Infrastructure can be software, a platform, or hardware that delivers or deploys applications to production.
- Infrastructure as code is a DevOps practice of automating infrastructure to achieve reliability, scalability, and security.
- The principles of IaC are reproducibility, idempotency, composability, and evolvability.
- By following the principles of IaC, you can improve change management processes, lower time spent on fixing failed systems in the long term, better share knowledge and context, and build security into your infrastructure.
- IaC tools include provisioning, configuration management, and image-building tools.

Workshop 3: Infrastructure Evaluation based on RICE Principles